Xilinx Toolchain and SSH HIL Testing

Training guide for the Zynq Sonar Platform

Audience: FPGA, embedded Linux, application, and test engineers. **Reference target:** Digilent Cora Z7-10 Rev. B, Zynq-7000. **Prepared:** October 23, 2026. **Suggested session:** 3 hours, plus unattended hardware and Linux builds.

Codex works through the project's installed AMD/Xilinx command-line tools, version-controlled Tcl, Linux build recipes, and SSH access to the board. This guide explains that integration and teaches a repeatable route from source changes to physical FPGA testing, regression evidence, and automation.

By the end, a learner should be able to:

- Explain the path from RTL and Tcl to XSA, PetaLinux, and a bootable board.
- Run host simulation separately from hardware-in-the-loop (HIL) tests.
- Use SSH to exercise the real DDR/DMA/FPGA/driver path and retain results.
- Define regression gates that detect functional and performance changes.
- Use a repeatable runner and describe how to extend it into scheduled CI.

Session map

Module	Activity	Typical training time
1-2	Integration model and lab preparation	25 minutes
3-4	Host verification and build workflow	30 minutes plus builds
5-7	SSH, FPGA loopback, accelerator checks	45 minutes
8-10	Deployment, regression, automation	45 minutes
11-13	Evidence, diagnosis, practical assessment	35 minutes

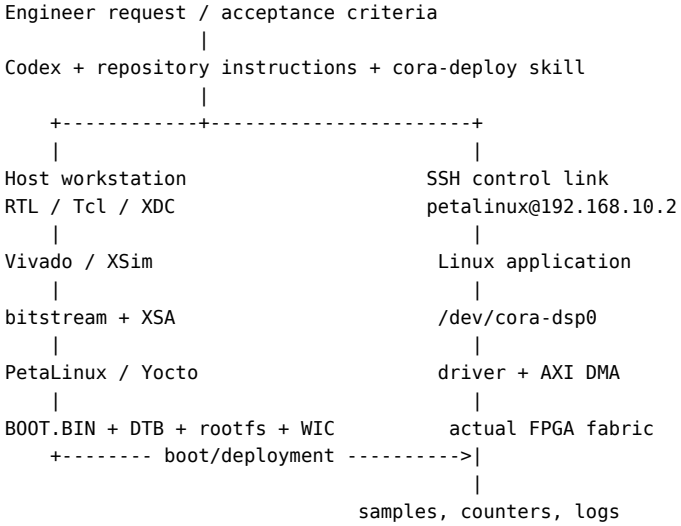
Evidence boundary

The commands and paths were reviewed against the working repository at commit `97c124f`, with existing local changes present. Historical board measurements are labeled with their dates. The read-only SSH probe on October 23 timed out; no live HIL test, deployment, build, reprogramming, or service restart was performed for this guide. The included runner is a training example, not an installed CI service.

Companion files: this editable Markdown guide and `examples/run-hil-regression.sh`, relative to this document. Execute examples from the repository root unless a command says otherwise.

1. How Codex integrates with the toolchain

The integration uses the same tools an engineer can run manually. Codex reads source and build results, edits the tracked inputs, invokes scripts, interprets failures, and can collect board-side evidence over SSH when the board is reachable and the work is authorized. The toolchain and test oracles determine success.



Integration point	Project implementation
Environment	`scripts/activate-tools.sh`
RTL simulation	`scripts/test-axis-pwm.sh`, `scripts/test-axis-acomms-router.sh`
Hardware construction	`hardware/vivado/tcl/create_cora_z7_10_pwm.tcl`
Hardware build entry	`scripts/build-pwm-hardware.sh`
Embedded Linux	`software/petalinux/cora-z7-10-baseline`
Shared recipes	`software/petalinux/meta-platform-common`
Remote operations	OpenSSH and the local `cora-deploy` skill/helper
Board test oracle	`cora-axis-loopback-test`

Vivado supplies synthesis, implementation, bitstream generation, XSA export, and XSim. PetaLinux builds Linux, device trees, drivers, applications, and boot artifacts. Vitis 2025.1 is available in the documented setup but is optional for these scripted Linux builds; no Vitis HLS or Vitis application flow is implied.

SSH controls programs on a booted Linux target. It does not provide cycle-level waveform visibility or recover a board that cannot boot. ILA, Vivado Hardware Manager, JTAG/XSDB, and serial-console recovery are separate workflows. See official Codex CLI documentation for the general local-tool execution model.

2. Prepare the host and reserve the board

Use the project's reproduced environment: Ubuntu 22.04.5, Vivado 2025.1, and PetaLinux 2025.1. These are the project baseline, not a recommendation to upgrade or a claim about the newest vendor release. The Cora device is `xc7z010clg400-1`; the registered board part is `digilentinc.com:cora-z7-10:part0:1.1`.

```
source scripts/activate-tools.sh
scripts/check-host.sh
vivado -version
printf 'PetaLinux %s\n' "$PETALINUX_VER"
python3 scripts/platformctl.py list
python3 scripts/platformctl.py validate cora-z7-10 --strict
```

Expected result: the 2025.1 tools resolve, the Cora platform is listed, and strict validation exits successfully. A registry validation checks metadata and configured paths; it does not test the physical board or certify a release bundle.

Bench checklist

- Boot the intended Cora SD image; provide power and Ethernet.
- Use the documented direct link: workstation `192.168.10.1/24`, board `192.168.10.2`.
- Make the dedicated SSH key available at `~/.ssh/cora_codex_ed25519`.
- Reserve the board and identify any process using `/dev/cora-dsp0`.
- Record intended image, hardware, device-tree, and driver versions.
- For electrical or acoustic testing, record wiring, instruments, gains, and fixture geometry.

The board runs a minimal PetaLinux image with SysV init and BusyBox tools. Use `/etc/init.d/<service>`, rather than assuming systemd. Do not assume the board provides `apt`, `install`, `flock`, or GNU command options. The regression runner keeps `flock` on the host and checks the board's `timeout` behavior before testing.

Hardware identity limitation

`docs/platform-identity.md` specifies a hardware identity/build-ID contract. The current Cora manifest still has null identity base address and build ID, and its artifact list lacks a complete DTB/release bundle. Do not teach that full hardware identity enforcement is already deployed. Existing driver/userspace core checks and controlled image provenance help, but cannot prove the entire active bitstream matches a manifest. A bitstream filename or FPGA-manager state alone cannot establish that identity.

Checkpoint: explain why a successful metadata validation cannot replace a board-side test.

3. Verify software and RTL on the host first

Start with deterministic tests that run without a board. They provide fast feedback before synthesis or bench use.

```
# Run each suite separately so Python module discovery stays isolated.
PYTHONPATH=software/cora-sonar \
  python3 -m unittest discover -s software/cora-sonar/tests -v

PYTHONPATH=software/cora-navigation:software/cora-ofdm-acoustic \
  python3 -m unittest discover -s software/cora-navigation/tests -v

python3 scripts/gnuradio-smoke-test.py
```

Install the dependencies documented for each component. The GNU Radio smoke script checks a four-item software multiply flowgraph; it does not access PWM, ADC, DMA, or FPGA hardware. Sonar and navigation software can process simulated scenes even when running on the board.

RTL lab

```
source scripts/activate-tools.sh
scripts/test-axis-pwm.sh
scripts/test-axis-acomms-router.sh
scripts/check-axis-pwm-synthesis.sh
```

The simulation scripts compile SystemVerilog with `xvlog`, elaborate with `xelab`, and run `xsim`. The scripts require their self-check PASS strings, in addition to successful commands. AMD documents this Vivado simulator batch flow in UG900.

Simulation	Required marker	Evidence directory
PWM core	`axis_pwm_s16 self-check: PASS`	`build/sim/axis_pwm_s16/`
PWM AXI wrapper	`axis_pwm_s16_axi self-check: PASS`	Same directory
Acoustic router	`axis_acomms_router self-check: PASS`	`build/sim/axis_acomms_router/`

The router testbench uses a stand-in accelerator that adds one to a sample. It verifies router behavior around that model; it is not a simulation of the installed FFT/filter/IFFT chain. Synthesis checks establish that the RTL can be synthesized, rather than that a routed board design meets timing.

Learner exercise: locate the router testbench's stand-in accelerator and explain which failures still require a real board: device-tree mapping, DMA/driver interactions, active image mismatch, and physical I/O behavior.

4. Build a coherent FPGA and Linux image

A. Generate and inspect the hardware

```
source scripts/activate-tools.sh
scripts/build-pwm-hardware.sh validate
scripts/build-pwm-hardware.sh build
```

`validate` constructs and validates the block design without implementation. `build` runs implementation through bitstream generation and exports an XSA containing the bitstream. The tracked Tcl is the source of truth; edits made only in the generated Vivado project must be reflected in Tcl to be reproducible.

Output	Location
Generated Vivado project	`build/vivado/cora-z7-10-pwm/cora_z7_10_pwm.xpr`
Hardware handoff	`hardware/export/cora-z7-10-pwm/cora-z7-10-pwm.xsa`
Timing/utilization/bus-skew reports	`build/reports/cora-z7-10-pwm/`

The Tcl checks implementation completion and rejects negative worst setup slack. Review the full timing report for hold, pulse-width, unconstrained-path, and clock issues, plus DRC results. A successful script exit alone does not establish every release criterion.

B. Import hardware and build Linux

```
scripts/fetch-meta-sdr.sh
petalinux-config \
  --project software/petalinux/cora-z7-10-baseline \
  --get-hw-description \
  "$PWD/hardware/export/cora-z7-10-pwm/cora-z7-10-pwm.xsa" \
  --silentconfig
scripts/build-petalinux-baseline.sh
```

The XSA handoff feeds hardware information into PetaLinux. Maintain matching addresses, interrupts, clocks, DMA properties, device-tree nodes, driver ABI, and userspace packages. AMD describes the import options in PetaLinux UG1144.

C. Package the boot image and SD image

```
cd software/petalinux/cora-z7-10-baseline
petalinux-package boot --force \
  --fsbl images/linux/zynq_fsbl.elf \
  --fpga images/linux/system.bit --u-boot \
  --output images/linux/BOOT.BIN
./package-wic.sh
```

The wrapper explicitly includes `system.dtb` in the WIC boot partition. Check image contents and freshly calculate SHA-256 values for release artifacts. Follow `docs/sd-image.md` for writing and booting a lab SD card. Preserve the previous known-good card/image. The older `scripts/build-petalinux.sh` uses a different project and base XSA; use the baseline flow above for the combined Cora design.

5. Use SSH as the test control interface

A. Probe the established lab connection

The helper belongs to this workstation's skill installation; it is not a repository dependency. A different workstation needs an equivalent SSH setup.

```
python3 ~/.codex/skills/cora-deploy/scripts/cora_deploy.py probe
```

Define a reusable Bash wrapper in the current host shell:

```
cora_ssh() {
  ssh -i "$HOME/.ssh/cora_codex_ed25519" \
    -o IdentitiesOnly=yes -o BatchMode=yes -o ConnectTimeout=8 \
    -o ServerAliveInterval=5 -o ServerAliveCountMax=2 \
    -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no \
    petalinux@192.168.10.2 "$@"
}
```

The host-key exception is the existing policy for this fixed, isolated, frequently re-imaged lab address. Use verified persistent host keys for other endpoints.

B. Capture a read-only preflight

```
mkdir -p results/training-preflight
cora_ssh 'id; hostname; uname -a; cat /proc/cmdline; cat /proc/uptime' \
  > results/training-preflight/identity.log 2>&1
cora_ssh 'ls -l /dev/cora-dsp0 /dev/cora-pwm0; ps' \
  > results/training-preflight/devices-processes.log 2>&1
cora_ssh 'dmesg | tail -n 100' \
  > results/training-preflight/kernel.log 2>&1
```

These inspection groups print evidence; they are not automated gates. Review every expected device and any permission or kernel-log error. For an assertion use an explicit check, for example `cora_ssh 'test -c /dev/cora-dsp0'`.

SSH normally returns the remote command's status; SSH transport failures generally return 255. A dropped connection does not prove the remote test stopped. Use a remote timeout, then inspect process/device state after failures. A host-side timeout alone only bounds the client.

Checkpoint: distinguish BLOCKED (board unreachable, wrong image, missing device, board already reserved) from FAIL (a validly configured test returns incorrect data). Today the guide preparation encountered the former: SSH connection timeout, with no board verdict.

6. Lab: exercise the physical FPGA loopback

The software supplies deterministic samples, and the real FPGA participates in the return path. This is digital HIL with synthetic stimulus.

```
Host --SSH command--> board test program
                          |
                 512 signed int16 samples
                          |
                 /dev/cora-dsp0 driver
                          |
                 DDR -> DMA MM2S
                          |
            AXIS router in loopback mode
                          |
                 DMA S2MM -> DDR
                          |
             compare samples + hardware counters
```

Run the shipped test

Ensure exclusive access to `/dev/cora-dsp0`. An idle dashboard page is not proof that its service has released the device. Identify the owner and coordinate its release; preserve unrelated services.

```
cora_ssh 'cora-axis-loopback-test --frames 32 --sample-rate 250000'
```

Expected output: `PASS Cora AXI-stream acoustic loopback`, 32 frames, 16,384 samples, byte-exact integrity, and the reported hardware rate/clock/divider. The default hardware is 100 MHz divided by 400, giving 250,000 samples/s.

Understand the oracle

- Every returned sample is compared with a deterministic signed-16 input frame.
- Hardware frame and sample counters must equal the requested totals.
- Elapsed time must be at least 95% of samples divided by the reported rate.
- A mismatch, bad counter, or implausibly fast completion exits with failure.

The current test repeats one seeded random frame for every transaction. That makes reproduction easy but can miss stale-frame reuse. Extend coverage with distinct per-frame patterns, sequence tags, zeros, extrema, and boundary cases when qualifying a new core. The existing test also lacks an upper runtime/throughput threshold and does not independently assert requested versus reported rate; add explicit gates for those requirements.

The router uses 32-bit AXIS words. The low 16 bits carry real data; the high 16 bits are the imaginary component, zero for real input. Frames contain 512 samples and terminate with `TLAST`. The driver presents 1,024-byte real-sample frames to userspace and handles DMA packing. This test validates digital transport; it does not measure an analog transducer or underwater channel.

7. Lab: verify the accelerator and optional physical I/O

After transport passes, test the processing chain independently:

```
cora_ssh 'cora-dsp-benchmark --frames 1000 --low-bin 8 --high-bin 24'
```

The shipped benchmark selects the accelerator path through `HwFftFilterDevice`. Its input has tones at bins 12 and 96; the 8-24 mask should preserve bin 12 and attenuate bin 96. The hardware uses a 512-point fixed-point FFT, symmetric spectral mask, and IFFT. The forward FFT scales by 1/512; the inverse is unscaled, giving approximately unity all-pass gain.

Quantity	What it tells the learner
RMSE and maximum error	Fixed-point output agreement with the CPU reference
Bin 96 attenuation	Whether the stopband behavior is correct
DMA failures	Transport/driver failures during the run
End-to-end latency	Application-visible transaction cost
DMA/core latency	Driver-recorded transaction time; not pure FFT compute time
CPU latency and speedup	Relative cost under the stated configuration/load

Critical distinction: the existing benchmark prints metrics but does not enforce numerical acceptance limits. Its exit code alone is not an accelerator correctness verdict. A regression extension must parse metrics, reject missing/nonfinite values, enforce zero DMA failures, and apply approved error, attenuation, and latency thresholds.

Start thresholds from a measured, reviewed reference distribution on the same image, clock, rate, stimulus, frame count, and load. For example, a candidate latency gate might be 20% above a baseline median; that is a proposed policy requiring calibration, not a measured project requirement. Fixed-point processing should use numerical tolerances rather than byte equality with floating point.

Additional bench coverage

- ADC/PWM: the documented `cora-adc-fft-pwm --low-bin 0 --high-bin 64` path needs the intended input wiring and scope/capture checks. Record stimulus, expected passband, output level, and measurements.
- USB audio: confirm playback/capture devices with `aplay -l` and `arecord -l`; perform offline modem checks before speaker-to-microphone tests. Record decoded payload hashes, errors, retries, SNR, gain, and geometry.
- Sonar/navigation scenes: running a synthetic scene on Cora validates target software execution, not a physical sonar or navigation sensor.

The current XADC source is polled through IIO and held between measurements. Its existence does not establish phase-continuous high-rate acquisition. Real multichannel receive testing requires the appropriate converter and DMA hardware.

8. Deploy changes and retain them in the image

Use a focused application deployment when validating a source or static-asset change. First identify its PetaLinux recipe, exact installed destination, and affected service. Do not assume all components use the same paths.

Example: a sonar dashboard asset

The established mapping is:

```
software/cora-sonar/www/index.html
  -> /usr/share/cora-sonar/www/index.html
service: /etc/init.d/cora-sonar
page: http://192.168.10.2:8085/
```

Inspect the local file, installed file/hash, and service state. For a required asset update:

```
python3 ~/.codex/skills/cora-deploy/scripts/cora_deploy.py deploy \
  software/cora-sonar/www/index.html \
  /usr/share/cora-sonar/www/index.html --mode 0644
```

The helper stages one file under a unique `/tmp` path, copies it with BusyBox-compatible tools, sets its mode, verifies SHA-256, and removes its staging file. Keep a recoverable copy of the prior asset before replacement. The sonar service reads this static file per request, so this example does not need a restart. For changed executable code, restart only the affected service if needed, then check its actual process and endpoint.

If the helper reports that sudo authentication is required, use its supported `--prompt-sudo` path interactively. Do not place passwords in scripts or command arguments. Automated runs need a separately provisioned noninteractive deployment policy; the training regression runner does not deploy.

Preserve the change for the next boot image

```
source scripts/activate-tools.sh
petalinux-build \
  --project software/petalinux/cora-z7-10-baseline -c cora-sonar
```

A focused recipe build validates packaging but does not refresh an existing WIC. Rebuild the full image and package it using module 4 before claiming that a replacement SD image contains the fix. Verify installed and served content, not merely copy success.

Changes to RTL, address maps, DMA, interrupts, or device-tree contracts require a coherent hardware/Linux bundle. This asset-copy example is not a bitstream update procedure. Prefer the documented matched SD-image path for training; hot programming requires a separate procedure for releasing devices and managing compatible drivers/device trees.

9. Design a regression suite with meaningful gates

Regression testing reruns comparable cases after changes and compares both correctness and measured behavior. Each case needs a stimulus, environment, oracle, timeout, and retained evidence.

Tier	Case	Acceptance	Suggested cadence
Host	Python component tests	Assertions pass; dependencies present	Each relevant change
RTL	PWM/router simulations	Exit success and expected self-check markers	RTL changes
Build	BD + implementation	Valid design; reviewed timing/DRC; coherent XSA	Hardware changes
Boot	Linux/device readiness	Correct bundle; devices, permissions, dependencies	New image
HIL transport	32-frame + repeated 2,048-frame loopback	Byte-exact data, exact counters, pacing check	Bench candidate
Accelerator	Tone/reference comparisons	Reviewed numeric limits and zero DMA failures	DSP/driver changes
Application	Modem payload transfer	Matching source/output hash; retry/error limits	Modem changes
Services	Feature start/stop and contention	Endpoint works; process/device really released	Init/service changes
Physical I/O	ADC, PWM, audio or sensor fixture	Calibrated electrical/acoustic limits	Fixture/release runs
Soak	Repeated transactions under fixed load	No drift, leaks, unexpected resets or errors	Scheduled lab run

Use three repeated baseline measurements before comparing throughput or latency. Add p50/p95 and worst-case timing when a richer harness captures per-frame durations. Keep functional gates separate from performance gates so a timing regression cannot disappear inside a data-integrity PASS.

Regression example from this project

The October 5 board report found that stopping the triplet service removed its PID file while leaving its Python process alive. That leftover load affected navigation timing. A corrected init script existed in source, and the replacement image still awaited reflash/on-off verification in that report. Teach a process-release check as well as a PID-file check; do not promote that source fix to a confirmed current-board result.

Result vocabulary

PASS: stated assertions passed under the recorded setup. **FAIL:** a valid test violated an assertion. **BLOCKED:** prerequisites or board access prevented a valid test. **SKIP:** the case intentionally does not apply, with a reason. Preserve first failures and all retries; a passing retry does not erase instability.

10. Automate the SSH loopback subset

The companion `examples/run-hil-regression.sh` turns existing board commands into a repeatable host-driven run. It is intentionally a small starting point that tests transport integrity; it does not implement the entire matrix in module 9.

Run it on a reserved, correctly provisioned board

```
bash docs/training/examples/run-hil-regression.sh
```

The host needs Bash, Git, GNU `timeout`, `flock`, OpenSSH, and `sha256sum`. The board needs its loopback utility, `/dev/cora-dsp0` access, and a `timeout` supporting the checked invocation. No service is stopped automatically. Coordinate release of any device owner before running.

The runner:

1. Creates a unique UTC-tagged evidence directory and acquires a host-local board lock.
2. Records commit, dirty status, tracked patch, and metadata hashes.
3. Checks SSH, device access, utility availability, and remote timeout support.
4. Runs 32 frames, then three independent 2,048-frame cases at 250,000 samples/s.
5. Requires both successful SSH/test completion and the exact loopback PASS marker.
6. Writes per-case logs and `summary.tsv`, then exits nonzero on blocked or failed work.

```
results/hil-training-<UTC>-<unique>/
  source-commit.txt       source-status.txt
  tracked-changes.patch   metadata-sha256.txt
  preflight.log           summary.tsv
  loopback-smoke.log      loopback-repeat-1.log
  loopback-repeat-2.log   loopback-repeat-3.log
```

A device timeout causes a fail-fast stop; inspect the board before issuing more DMA requests. A remote timeout is useful but cannot reliably kill a kernel task stuck in uninterruptible I/O. Verify process/device state after a failure. This example captures tracked changes, but not the contents of arbitrary untracked files; use the source snapshot workflow in module 11 for selected inputs.

Extend into CI or scheduled runs

Use a lab runner with the pinned toolchain and board network access. Serialize jobs per board across all hosts using a shared board lease; the included local lock only coordinates cooperating runs on one host. Run host tests first, build immutable artifacts, provision a matched bundle in a dedicated stage, wait for readiness with a bounded retry policy, execute tests sequentially, and publish artifacts even after failure.

Add accelerator metric parsing, numerical thresholds, JUnit/JSON export, and a board recovery policy before treating this as release CI. A scheduler can invoke the fixed script nightly; no schedule, CI account, notification, or autonomous deployment was installed by preparing this guide.

11. Retain evidence and interpret measurements

A useful run record identifies the source, exact artifacts, target, stimulus, load, assertions, and raw results. Retain commit plus dirty/untracked input provenance; tool versions; XSA, bitstream, DTB, boot/rootfs hashes; kernel and device information; test versions; rate/frame/seed settings; stdout/stderr; exit codes; and timing.

The project can archive selected source inputs and optional live-board state:

```
python3 scripts/platformctl.py snapshot cora-z7-10 \
  --label pre-regression --live \
  --ssh-key "$HOME/.ssh/cora_codex_ed25519"
```

This writes JSON and a source archive under `platforms/cora-z7-10/snapshots/`. Inspect each recorded live command's return code; the snapshot command can complete even when individual SSH commands failed. The archive covers configured include patterns, not the entire workstation or every release artifact. A snapshot is evidence capture, not hardware identity verification.

Historical case study: September 15, 2026

Measurement	Recorded value	Interpretation
FPGA loopback length	2,048 frames / 1,048,576 samples	Byte-exact transport result
FPGA clock/divider	100 MHz / 400	Configured 250,000 samples/s
Application throughput	148,497.9 samples/s	Includes per-frame userspace/DMA overhead
Modem through PL loopback	16,384-byte payload, 43 packets	Actual digital PL transport in the channel
Payload goodput	3,011.57 bit/s	Includes modem/framing/software cost
Payload integrity	Matching source/output SHA-256	Zero retries and packet errors recorded

For 2,048 frames, ideal paced sample time is `1048576 / 250000 = 4.194304 s`. The recorded application throughput corresponds to roughly 7.06 seconds. Inter-frame transactions and software overhead explain why application throughput is lower than configured PL pacing; those are different metrics.

These values come from `docs/benchmark-results/cora-modem-summary-2026-09-15.json`. They are historical observations, not freshly measured results or automatic thresholds. The same summary explicitly states that no USB soundcard was present during its summary run; its USB audio results were drawn from earlier tests. Its underwater projections are estimates and must not be cited as physical underwater HIL evidence.

The existing manifest also records an older, dirty source revision. Compare and refresh provenance for a new release rather than assuming a stored manifest proves the current working tree or active board image.

12. Diagnose failures by layer

Preserve the failing log before retrying. Change one variable at a time and rerun the smallest relevant case.

Symptom	First checks	Next action
SSH connection timeout	Power, Ethernet, route, IP/subnet, boot progress	Use serial console; classify access as BLOCKED
SSH authentication failure	User, dedicated key, identities policy	Repair credentials without logging secrets
`/dev/cora-dsp0` absent	Kernel log, module probe, device tree, image	Reconcile matched build before DMA
Permission denied	Device owner/mode, groups, udev rule	Apply intended access policy
Device busy	Test reservation, process list, open device owner	Coordinate only the affected owner
DMA timeout or frame never returns	TLAST, AXIS ready/valid, resets, clocks, IRQ/DT	Compare loopback with accelerator path
Loopback passes; FFT result wrong	Filter bins, fixed-point scaling, packing, core configuration	Compare deterministic tones to CPU reference
Data passes; throughput drops	CPU load, frame size, rate/divider, scheduler	Repeat under recorded baseline load
Stop reports success; CPU stays busy	Actual process command/PID and device ownership	Verify process exit, beyond PID-file removal
Live fix disappears after reflash	Recipe inputs and rebuilt rootfs/WIC	Package and verify a new coherent image

Use the loopback switch to narrow the fault

If the router's byte-exact path passes while the accelerator path fails, basic DMA/driver transport is working under that setup. Investigate accelerator scaling, algorithm, framing, and handshake behavior. If both fail, begin with the shared transport, clocks/resets, interrupts, driver, device tree, and active image. This narrows the investigation; it is not proof that every shared component is flawless.

Recover deliberately

Do not blanket-stop unrelated Cora services or repeatedly hammer a timed-out DMA engine. Record processes and kernel diagnostics, confirm the test stopped, and use the established recovery procedure for the affected component. If Linux/SSH is unavailable, use serial/JTAG recovery and restore a known-good matched image. After recovery, repeat preflight and the minimal transport test before longer regressions.

Learner exercise: a 32-frame loopback succeeds, but navigation timing falls when triplet runs. Classify the problem, list the evidence needed, and explain why rebuilding the FFT is not the first diagnostic step.

13. Practical assessment and working with Codex

Instructor-led capstone

1. Identify the target, source revision, dirty inputs, tool versions, and intended image. Explain the current hardware identity limitation.
2. Run a host software test and router RTL simulation. Retain the exit codes and correct self-check marker.
3. On an available reserved board, collect preflight evidence and run the 32-frame loopback. Explain the expected 16,384 samples.
4. Run the companion regression example. Inspect every completed case, all failure logs, and the limits of its final PASS.
5. Run the spectral benchmark and explain why printed metrics need an additional threshold evaluator before becoming a correctness gate.
6. Compare throughput with a recorded baseline only after matching image, frame count, rate, stimulus, and load.
7. Present one evidence bundle with separate transport, accelerator, application, and physical-I/O conclusions.

Pass criteria: the learner reproduces the commands, interprets scope correctly, retains enough evidence to investigate a failure, and never presents a blocked test, simulated sensor, or historical measurement as a current physical test result.

Useful instructions to give Codex

Inspect and explain

Trace the current Cora Vivado Tcl, XSA handoff, PetaLinux recipes, driver ABI, and loopback test. Show which pieces are implemented and which compatibility checks remain incomplete.

Run a focused HIL regression

On the reserved Cora board, collect a preflight snapshot and run the existing AXIS loopback at 32 frames and three repeats of 2048 frames at 250000 samples/s. Preserve logs and exit codes. Report blocked prerequisites separately from data failures. Preserve unrelated services.

Build and investigate a DSP change

Update the tracked RTL/Tcl, run focused simulation, implement the Cora design, and review timing. Build a matched PetaLinux candidate. Report the XSA, bitstream and device-tree hashes, then define board checks against the CPU reference with explicit tolerances.

Diagnose a regression

Compare these two evidence bundles. Check artifact and test configuration equality first, then separate integrity, numerical accuracy, and performance changes. Identify the smallest test that can distinguish the likely causes.

Always request a result grounded in commands, artifacts, and assertions. Codex can carry out the engineering workflow through the available tools, while deterministic test programs and measured acceptance criteria make the result reproducible without an AI session.

Reference shelf and capability status

Project sources used to prepare this guide

Source path, relative to repository root	Purpose
`README.md`, `docs/build-host.md`, `docs/version-matrix.md`	Baseline tools and platform scope
`scripts/activate-tools.sh`, `scripts/build-pwm-hardware.sh`	Tool activation and Vivado entry points
`hardware/vivado/tcl/create_cora_z7_10_pwm.tcl`	Actual hardware construction and build checks
`scripts/test-axis-pwm.sh`, `scripts/test-axis-acomms-router.sh`	Existing RTL simulations
`scripts/build-petalinux-baseline.sh`	Active Linux project build
`software/petalinux/cora-z7-10-baseline/package-wic.sh`	DTB-inclusive image packaging
`docs/axis-acomms-loopback.md`, `software/cora-dsp/README.md`	Digital HIL path and ABI
`software/cora-pwm/gnuradio/gr-cora/examples/cora-axis-loopback-test`	Actual integrity/counter/pacing assertions
`software/cora-pwm/gnuradio/gr-cora/examples/cora-dsp-benchmark`	Accelerator metrics and reference computation
`docs/platform-identity.md`, `platforms/cora-z7-10/platform-manifest.json`	Desired identity contract and current gaps
`docs/cora-combined-demo.md`	October 5 board checks and deployment caveats
`docs/benchmark-results/cora-modem-summary-2026-09-15.json`	Historical measured examples
`scripts/platformctl.py`, `docs/sd-image.md`	Snapshots and SD-image operations
`~/.codex/skills/cora-deploy/SKILL.md`	Workstation-specific SSH/deployment procedure

Capability status as reviewed for this edition

Present in the project: pinned tool activation, Tcl-based Vivado builds, XSim self-check scripts, XSA/PetaLinux handoff, Linux DMA device interface, selectable digital loopback/FFT path, board-side test commands, shared recipes, and source/live snapshot tooling.

Historically demonstrated: Cora boot, byte-exact FPGA loopback, spectral benchmark, and application/modem tests documented in the cited September and October project records. Cora evidence must not be generalized to the UltraZed target; its documented implementation has not been boot-validated on that hardware.

Provided with this guide: a host-driven loopback regression example with logs, time bounds, local serialization, and a limited integrity verdict. It was syntax-checked locally, not executed against the unreachable board.

Extensions requiring implementation/qualification: distributed board leasing, complete hardware identity/build-ID enforcement, approved accelerator/performance gates, richer frame-sequence coverage, JUnit/JSON CI reporting, unattended image provisioning/recovery, and scheduled regression infrastructure.

Official reference links are embedded in modules 1, 3, and 4. All project paths above refer to reviewed local source; consult the current files before executing a lab after subsequent changes.